

Design Spec — Claude Code Workshop (6 hours, hands-on)

Date: 2026-06-19 **Author:** Uroš Godnov (with Claude Code) **Status:** Draft for review

1. Summary

A standalone, professional, **6-hour hands-on workshop** that teaches **non-technical professionals** to work productively with **Claude Code** (Terminal CLI), and — as a second agentic cockpit — **Google Antigravity** with MCP. The day is organised around a single business deliverable that grows from a raw sales CSV into a shipped, self-running sales-report tool. Every Claude Code capability the brief calls out — the **agentic loop**, **goals/plan mode**, **reasoning efforts**, **token consumption across settings**, **MCP servers**, **GitHub integration**, the **/loop automation command**, and **Antigravity + MCP** — is introduced at the moment the project needs it.

Deliverables: a Quarto/reveal.js slide deck, a pre-executed token-consumption Jupyter notebook, six copy-paste exercise cards, deterministic fallback artifacts, the Adriatica orders.csv, instructor notes, a pre-work sheet, and a README — all under a new `claude-code-workshop/` directory. Language: English.

This workshop is **outside** the IMB2026 student course's hard "free-tools-only" constraint: it is a standalone professional workshop and assumes participants have paid Claude Code access. It therefore lives in its own top-level directory, not under `workshops/lab-aNN/`.

2. Audience and context

- **Who:** non-technical professionals (managers/analysts who code little or not at all).
- **Engagement:** hands-on. Participants run real Claude Code on their own machines.
- **Primary surface:** **Terminal CLI** (the canonical, most-documented Claude Code surface).
- **Setup:** guided, in the first block. A lightweight optional pre-work sheet de-risks it.
- **Access/cost:** paid Claude access is assumed and acceptable (not a free-tools workshop).
- **Language:** English (matches the IMB2026 programme language).

Core pedagogical bet. The Terminal CLI is historically intimidating, but for a non-technical audience the resolution is pedagogical, not technical: once a participant types `claude` and hits enter, the terminal stops being a place where *they* type cryptic commands and becomes a place where they *have a conversation* and the agent runs the commands. The workshop front-loads all terminal/install fear into one supported block, then never asks them to fear it again. This mirrors the IMB2026 course principle "the AI writes the code; the learner specifies, reviews, verifies" (`planning/workshop-plans/lab-a05-vibe-coding-project.md`), applied to `ops/git` instead of `pandas`.

3. Goals (learning objectives)

After the workshop a participant can:

1. Install, authenticate, and open Claude Code in a terminal, and hold a working conversation.
2. Explain the **agentic loop** (gather context → act → verify → repeat) in plain language and recognise it happening on screen.
3. Give Claude a **goal**, use **plan mode**, and review/approve actions before they run.
4. Change **reasoning effort**, switch **models**, and use `/fast`, and feel the speed/quality/cost trade-off.
5. Read and reason about **token consumption** under different settings (model, effort, caching, context size) using `/cost`, `/context`, `/compact`, and the measurement notebook.
6. Explain what an **MCP server** is and add one to Claude Code.
7. Direct Claude to perform a **GitHub** workflow (create a repo, open a pull request) without typing git commands themselves.
8. Start and approach a **project in Antigravity** with an MCP server connected.
9. Use the `/loop` command (and scheduled/background work) to make a task recur.
10. Apply the **verify-everything** habit: running ≠ correct; check agent output against a known value.

3a. Non-goals

- Not teaching Python, pandas, or git *syntax*. Participants direct; the agent writes/runs code.
 - Not an API-engineering course. The notebook uses the Anthropic SDK only to *measure* token usage; participants read results, they do not build software with the SDK.
 - Not a comprehensive Claude Code reference. Scope is the brief's named capabilities, taught through one project.
-

4. Approach (chosen)

Spine-driven (“one business project, growing capabilities”). A single tiny business scenario — *turn orders.csv into a monthly sales report, then ship and automate it* — runs the whole day. Each capability appears because the project needs it. Chosen over a feature-tour (low retention for non-technical) and an open-sandbox (too loose for a paid day), and consistent with the IMB2026 “business-question-first / shared dataset spine” design DNA.

Dataset: reuse the Adriatica retail orders.csv already in the repo (workshops/lab-a01-pandas-business-data/data/orders.csv), copied into the workshop's data/ for self-containment.

Robustness commitments:

- **Deterministic fallbacks** — every live-AI step ships with a pre-written “known-good” artifact so a flaky generation or exhausted quota never strands the room (pattern borrowed from the A05 planted-artifact design).
 - **Optional pre-work** — a 1-page account+Node sheet de-risks the guided setup block.
-

5. Block design (sequence fixed; durations are instructor discretion)

Timing note. Block *order* is the design. Block *durations*, breaks, and lunch placement are the instructor's discretion and live only in the instructor notes as a suggested, adjustable template — not hard-coded into the deck. A 90-minute lunch around the mid-point creates a natural teaching seam: *turn the dials* (effort/model) before lunch, *read the meter* (token cost) after.

Block 1 — Doorway & Setup

Guided install + authentication; open a terminal; run `claude`; first conversation (“summarise what's in this folder”). Goal: every participant has a working Claude Code and has felt the terminal become a conversation. Highest-risk block → most facilitator support, troubleshooting playbook, and a “pair up / watch the demo” fallback.

Block 2 — The Loop & The Goal

The **agentic loop** and **goals / plan mode**, taught on a felt task: “analyse orders.csv and give me a monthly sales summary.” Claude writes and runs the Python; the participant sets the goal, watches the loop, and **approves each action** (permission modes). Introduces the review reflex. Fallback: `fallbacks/b2-sales-summary.py` + expected output.

Block 3a — Turning the Dials (*pre-lunch*)

Reasoning efforts (low → high → max), **model choice** (Opus/Sonnet/Haiku), and `/fast`. Re-run the Block 2 task under different settings; feel the speed/quality difference. Sets up the cost question answered after lunch.

Block 3b — Reading the Meter (*post-lunch*)

Token consumption across settings, made concrete with the **pre-executed notebook** (model × effort × caching × context size → tokens, cost, latency) plus in-CLI `/cost`, `/context`, `/compact`, `/clear`. The “what did the dials actually cost” payoff.

Block 4 — Giving Claude Hands (MCP + GitHub)

What an **MCP server** is and why ("new senses and hands for the agent"); add one to Claude Code (`claude mcp add ...`). Then **GitHub**: connect via `gh` and/or the GitHub MCP server and direct Claude to create a repo and **open a pull request** with the report — the emotional peak, a non-coder shipping a real PR without typing `git`. Fallback: `fallbacks/b4-pr-example.md`, `fallbacks/b4-mcp-config.*`.

Block 5 — A Second Cockpit: Antigravity

The MCP concepts transfer: install **Google Antigravity**, connect an MCP server, and approach the same project as a project in a visual agentic IDE. Shows MCP is a standard, not a Claude quirk, and leaves participants a friendlier IDE. Budget hands-on steps for Antigravity's tight (weekly) free quota; fallback to a recorded/demo path if quota is exhausted.

Block 6 — Make It Run Itself + Wrap

The `/loop` command and scheduled/background work — make the report "run every Monday." Cost & safety habits, the verify-everything refrain, capstone recap (CSV → analysed → shipped → automated), and where to go next.

6. Deliverables and file layout

New top-level directory (standalone; outside the free-tools student track):

```
claude-code-workshop/
├── README.md                # facilitator overview, schedule template, prerequisites, cost budget
├── pre-work.md              # 1-page participant insurance sheet (account + Node before the day)
├── slides/
│   └── claude-code-workshop.qmd # Quarto -> reveal.js deck, all 6 blocks
├── notebook/
│   └── token-consumption-lab.ipynb # PRE-EXECUTED presentation notebook (Anthropic SDK + charts)
├── exercises/
│   ├── b1-setup.md
│   ├── b2-loop-and-goal.md
│   ├── b3-efforts-and-tokens.md
│   ├── b4-mcp-and-github.md
│   ├── b5-antigravity.md
│   └── b6-loop-automation.md
├── fallbacks/                # deterministic "known-good" artifacts per live-AI step
│   ├── b2-sales-summary.py
│   ├── b2-expected-output.md
│   ├── b4-pr-example.md
│   ├── b4-mcp-config.json
│   └── b5-antigravity-mcp-config.json
├── data/
│   └── orders.csv            # copy of Adriatica retail data (self-contained)
└── instructor-notes.md      # durations template, troubleshooting, Antigravity quota caveat, fallback
```

6.1 Slide deck (`slides/claude-code-workshop.qmd`)

- Quarto → reveal.js. Title section + six block sections mirroring §5.
- Each block alternates **concept slides** (short, business-framed) with "□ **Your turn**" slides that show the *exact prompt to copy*. Speaker notes carry the facilitator script.
- Renders to a self-contained HTML deck.

6.2 Token-consumption notebook (notebook/token-consumption-lab.ipynb)

- **Presentation notebook:** authored, executed once, and **committed with outputs**, so participants *read* measured numbers without needing an API key. A clearly-marked optional cell allows re-running with one's own key.
- **Measures** the same business prompt under: (1) model — Opus/Sonnet/Haiku; (2) reasoning effort — low→max; (3) prompt caching — cached vs uncached; (4) context size — small vs stuffed. Reads `response.usage` (input/output/cache tokens) each time; computes \$ cost and latency.
- **Output:** one tidy results table + 2–3 bar charts (cost-per-model, cost-per-effort, cached-vs-uncached).
- **Accuracy:** current model IDs and per-token pricing pulled from the `claude-api` reference **at build time** (not from memory).
- **Build cost:** a handful of tiny calls (a few cents), documented in instructor notes.

6.3 Exercise cards (exercises/b*.md)

Each hands-on step is a card: one-line **business goal**, the **exact text to type into Claude**, **what you should see**, and an **“if it looks wrong, say this”** recovery line. No participant invents a command or a git incantation.

6.4 Fallbacks (fallbacks/)

Pre-written known-good artifacts for each live-AI step; instructor notes say exactly when to reach for each.

6.5 Instructor notes (instructor-notes.md)

Suggested durations template (instructor discretion) + natural break/lunch points; Block 1 setup troubleshooting (Node/npm, auth, Windows vs Mac, pair/watch fallback); Antigravity weekly-quota caveat with a **re-verify-1-week-before** reminder; realistic per-participant token cost budget for the day; fallback cues; verify-everything refrain.

6.6 Pre-work sheet (pre-work.md)

Optional 1-page doc to send beforehand: create the account, install Node — de-risks guided Hour 1.

7. Accuracy and “latest versions” notes

The brief asks to reflect the latest Claude Code. At build time:

- Consult the `claude-api` skill/reference for current model IDs, pricing, and token/usage fields used by the notebook — do not recite from memory.
- Verify current Claude Code surface details (effort tiers, `/fast`, `/loop`, MCP add syntax, `/cost` `/context` `/compact`) against the running CLI and official docs (<https://docs.claude.com/>).
- Re-verify **Antigravity** free-tier limits ~1 week before delivery (per `planning/resources.md` and memory: quota moved to weekly; no committed permanent free tier).

8. Risks and mitigations

Risk	Mitigation
Setup eats the morning (non-technical + terminal)	Guided block with troubleshooting playbook; optional pre-work sheet; pair
Live AI generates wrong/odd output	Deterministic fallback artifact for every live step
Antigravity free quota exhausted mid-room	Budget steps for scarcity; recorded/demo fallback; re-verify limits before de
Token/pricing facts go stale	Pull from <code>claude-api</code> at build; notebook outputs dated; verify before deliv
Windows/Mac divergence in setup	Both paths in instructor notes; facilitator demo on the majority OS
“6 hours” interpreted differently	Durations are an adjustable template under instructor discretion, not fixed

9. Open items to confirm before/at build

- Exact business framing copy for the running scenario (report contents, the “known number” used for verification — can reuse A01 anchors: total revenue, best month).
- Which specific MCP server(s) to demo in Block 4/5 (e.g., filesystem, GitHub, a docs server) — decided at build against what installs cleanly and freely.
- Whether to ship the optional pre-work sheet to participants (instructor's call).